template.typ — Reference & Usage Guide

linalg/docs Aren Vista ·

August 5, 2026

Contents

1 Traits

`NumericTraits<T>` is the scalar-type abstraction the rest of the library queries instead of assuming `double`. Each method below is `static` and exists for both the real primary template and the `std::complex<T>` specialization, which share the same contract; where the two differ, the difference is noted. Throughout, `Scalar` is `T`, and `Real` is the magnitude type — equal to `Scalar` for real `T`, and the component type for complex `T`.

`static Real epsilon()`

Example

```
assert(NumericTraits<double>::epsilon() ==
       std::numeric_limits<double>::epsilon());
// component type for complex: Real is double
assert(NumericTraits<std::complex<double>>::epsilon() ==
       std::numeric_limits<double>::epsilon());
```

Description

Machine epsilon: the gap between 1 and the next representable `Real`, i.e. $\varepsilon := \min\{\delta > 0 : \mathrm{fl}(1+\delta) \neq 1\}$. For a complex `T` it is the epsilon of the component type — it characterizes the floating-point format, not the complex number.

Returns

`std::numeric_limits<Real>::epsilon()` as `Real`.

`static Real safeMin()`

Example

```
double sm = NumericTraits<double>::safeMin();
assert(sm > 0.0);
assert(std::isfinite(1.0 / sm)); // reciprocal never overflows
```

Description

Smallest positive value s such that $1/s$ is finite — the scaling threshold used to avoid overflow (LAPACK `dlamch('S')`). On IEEE formats this reduces to `std::numeric_limits<Real>::min()`; the guard covers formats where $1/\min()$ would overflow.

Returns

The safe minimum as `Real`; always positive with a finite reciprocal.

`static Real abs(const Scalar& x)`

Example

```
assert(NumericTraits<double>::abs(-3.5) == 3.5);
// complex modulus: |3 + 4i| = 5
assert(NumericTraits<std::complex<double>>::abs({3, 4}) == 5.0);
```

Description

Magnitude of a scalar, $|x|$. For real `T` this is `std::abs` from `<cmath>` (chosen to avoid signed-zero issues); for complex `T` it is the modulus, computed via `std::abs` (hypot-based, so $\mathrm{re}^2 + \mathrm{im}^2$ cannot overflow prematurely).

Parameters

`x` the scalar whose magnitude is taken

Returns

$|x|$ as `Real`.

`static Real absSquared(const Scalar& x)`

Example

```
assert(NumericTraits<double>::absSquared(-4.0) == 16.0);
// |3 + 4i|^2 = 25, computed without a square root
assert(NumericTraits<std::complex<double>>::absSquared({3, 4}) == 25.0);
```

Description

Squared magnitude $|x|^2$, computed without a square root — so it is exact for representable squares. Real `T` returns $x \cdot x$; complex `T` returns `std::norm(x)` $= \mathrm{re}^2 + \mathrm{im}^2$.

Parameters

`x` the scalar whose squared magnitude is taken

Returns

$|x|^2$ as `Real`.

`static Scalar conj(const Scalar& x)`

Example

```
assert(NumericTraits<double>::conj(-7.25) == -7.25);        // identity
assert(NumericTraits<std::complex<double>>::conj({1, 2}) ==
       std::complex<double>(1, -2));                        // 1 - 2i
```

Description

Complex conjugate $\overline{x}$. For real `T` this is the identity; for complex `T` it flips the sign of the imaginary part. Involutive: $\overline{\overline{x}} = x$.

Parameters

`x` the scalar to conjugate

Returns

$\overline{x}$ as `Scalar`.

`static Real real(const Scalar& x)`

Example

```
assert(NumericTraits<double>::real(-7.25) == -7.25);
assert(NumericTraits<std::complex<double>>::real({1, 2}) == 1.0);
```

Description

Real part $\Re(x)$. For real `T` this returns `x` unchanged; for complex `T` it returns `x.real()`.

Parameters

`x` the scalar whose real part is taken

Returns

$\Re(x)$ as `Real`.

`static Real imag(const Scalar& x)`

Example

```
assert(NumericTraits<double>::imag(-7.25) == 0.0);          // zero for real T
assert(NumericTraits<std::complex<double>>::imag({1, 2}) == 2.0);
```

Description

Imaginary part $\Im(x)$. Always zero for real `T`; for complex `T` it returns `x.imag()`.

Parameters

`x` the scalar whose imaginary part is taken

Returns

$\Im(x)$ as `Real`.

`static Scalar sqrt(const Scalar& x)`

Example

```
assert(NumericTraits<double>::sqrt(4.0) == 2.0);
// principal branch: sqrt(-1) = +i, not -i
auto r = NumericTraits<std::complex<double>>::sqrt({-1, 0});
assert(std::abs(r - std::complex<double>(0, 1)) < 1e-12);
```

Description

Principal square root $\sqrt{x}$. Real `T` uses `std::sqrt`; complex `T` uses the principal branch (`std::sqrt`), so the result has non-negative real part and squaring it recovers the input.

Parameters

`x` the scalar whose principal square root is taken

Returns

$\sqrt{x}$ as `Scalar`.

`static Scalar zero()`

Example

```
assert(NumericTraits<double>::zero() == 0.0);
assert(NumericTraits<std::complex<double>>::zero() ==
       std::complex<double>(0, 0));
```

Description

The additive identity for the scalar type — `Scalar(0)`, i.e. $0 + 0i$ for complex `T`.

Returns

Zero as `Scalar`.

`static Scalar one()`

Example

```
assert(NumericTraits<double>::one() == 1.0);
assert(NumericTraits<std::complex<double>>::one() ==
       std::complex<double>(1, 0));
```

Description

The multiplicative identity for the scalar type — `Scalar(1)`, i.e. $1 + 0i$ for complex `T`.

Returns

One as `Scalar`.

`static bool isApproxZero(const Scalar& x, Real tol)`

Example

```
assert(NumericTraits<double>::isApproxZero(1e-15, 1e-12));
assert(!NumericTraits<double>::isApproxZero(1e-10, 1e-12));
// complex: judged by modulus, |3e-3 + 4e-3 i| = 5e-3
assert(NumericTraits<std::complex<double>>::isApproxZero({3e-3, 4e-3}, 6e-3));
```

Description

Tests whether a scalar is negligible against an absolute tolerance, returning $|x| \le \text{tol}$. The test is by magnitude, so a purely imaginary value is not "zero" merely because its real part is.

Parameters

`x` the scalar to test
`tol` absolute magnitude tolerance (a `Real`)

Returns

`true` if $|x| \le \text{tol}$, otherwise `false`.

2 Exceptions

The library reports every contract violation by throwing a type derived from `LinalgError`, which itself derives from `std::runtime_error`. A single `catch (const LinalgError&)` therefore catches everything the library throws, while the specific subclasses carry structured detail (offending indices, pivot positions, iteration counts) for callers that want it. In an ordered `try` / `catch`, the specific handler wins over the `LinalgError` base.

2.1 LinalgError

`LinalgError(const std::string& message)`

Example

```
LinalgError e("something broke");
assert(std::string(e.what()) == "something broke");
// it is a std::runtime_error
try { throw LinalgError("msg"); }
catch (const std::runtime_error& re) { assert(std::string(re.what()) == "msg"); }
```

Description

Root of the exception hierarchy. Thrown directly by unimplemented stubs and for one-off contract violations that have no dedicated subclass; every other library exception derives from it.

Parameters

`message` human-readable description of the failure

Returns

Constructs the error; `what()` returns `message`.

2.2 DimensionMismatch

`DimensionMismatch(size_t lhsRows, size_t lhsCols, size_t rhsRows, size_t rhsCols)`

Example

```
DimensionMismatch e(3, 4, 5, 6);
assert(e.lhsRows() == 3 && e.lhsCols() == 4);
assert(e.rhsRows() == 5 && e.rhsCols() == 6);
```

Description

Thrown when operand shapes are incompatible for the requested operation. Vectors report their size as an $n \times 1$ shape. Records both operands' shapes so the handler can report exactly what failed to line up.

Parameters

`lhsRows`	rows of the left operand
`lhsCols`	columns of the left operand
`rhsRows`	rows of the right operand
`rhsCols`	columns of the right operand

Returns

Constructs the error; `what()` names all four dimensions.

`size_t lhsRows() / lhsCols() / rhsRows() / rhsCols() const`

Example

```
try { throw DimensionMismatch(2, 2, 7, 1); }
catch (const LinalgError& le) {
    auto* dm = dynamic_cast<const DimensionMismatch*>(&le);
    assert(dm && dm->rhsRows() == 7); // detail survives a base-ref catch
}
```

Description

The four recorded operand dimensions. They remain accessible after the exception is caught by base reference and `dynamic_cast` back down.

Returns

The corresponding recorded row/column count as `size_t`.

2.3 IndexOutOfRange

`IndexOutOfRange(size_t index, size_t bound)`

Example

```
IndexOutOfRange e(9, 4);
assert(e.index() == 9 && e.bound() == 4);
```

Description

Thrown by the checked accessors (`at`, checked element access) when an index is not less than its bound. The unchecked `operator()` never throws this.

Parameters

`index`	the index that was requested
`bound`	the exclusive upper bound; valid indices are `< bound`

Returns

Constructs the error; `what()` names the index and bound.

`size_t index() / bound() const`

Example

```
IndexOutOfRange e(9, 4);
assert(e.index() == 9);  // offending index
assert(e.bound() == 4);  // exclusive upper bound
```

Description

The offending index and the exclusive bound it violated.

Returns

The recorded index / bound as `size_t`.

2.4 SingularMatrix

`SingularMatrix(size_t pivotIndex)`

Example

```
SingularMatrix e(2);
assert(e.pivotIndex() == 2);
// thrown by factorizations/solves that hit a zero pivot, e.g.
try { (void)Matrix<double>{{1, 2}, {2, 4}}.inverse(); }
catch (const SingularMatrix&) { /* rank-deficient */ }
```

Description

A factorization or solve met an exactly (or numerically) zero pivot. `pivotIndex` is the elimination step that failed.

Parameters

`pivotIndex` the elimination step whose pivot was (near) zero

Returns

Constructs the error; `what()` contains "singular" and the pivot index.

`size_t pivotIndex() const`

Example

```
SingularMatrix e(2);
assert(e.pivotIndex() == 2);
```

Description

The elimination step at which the (near-)zero pivot appeared.

Returns

The recorded pivot index as `size_t`.

2.5 NotPositiveDefinite

`NotPositiveDefinite(size_t pivotIndex)`

Example

```
NotPositiveDefinite e(1);
assert(e.pivotIndex() == 1);
```

Description

A Cholesky-family factorization met a non-positive pivot; `pivotIndex` is the first such pivot. Only raised when the caller opted into `throwOnIndefinite`.

Parameters

`pivotIndex` index of the first non-positive pivot

Returns

Constructs the error; `what()` contains “positive definite”.

`size_t pivotIndex() const`

Example

```
NotPositiveDefinite e(1);
assert(e.pivotIndex() == 1);
```

Description

The index of the first non-positive pivot encountered.

Returns

The recorded pivot index as `size_t`.

2.6 ConvergenceFailure

`ConvergenceFailure(const std::string& algorithm, size_t iterations)`

Example

```
ConvergenceFailure e("GMRES", 500);
assert(e.algorithm() == "GMRES");
assert(e.iterations() == 500);
```

Description

An iterative algorithm exhausted its iteration budget. Solvers that report convergence through `converged()` / a `Report` object do not throw this.

Parameters

`algorithm` name of the algorithm that failed to converge
`iterations` number of iterations performed before giving up

Returns

Constructs the error; `what()` names the algorithm and iteration count.

`const std::string& algorithm() / size_t iterations() const`

Example

```
ConvergenceFailure e("GMRES", 500);
assert(e.algorithm() == "GMRES" && e.iterations() == 500);
```

Description

The algorithm name and the number of iterations performed before failure.

Returns

The recorded algorithm name (`const std::string&`) / iteration count (`size_t`).

2.7 NotComputed

`NotComputed(const std::string& factorization)`

Example

```
NotComputed e("QR");
assert(std::string(e.what()).find("QR") != std::string::npos);
assert(std::string(e.what()).find("compute()") != std::string::npos);
```

Description

A factorization accessor (`solve` , `factors` , `eigenvalues` , …) was called before `compute()` , or after a `compute()` that failed.

Parameters

`factorization` name of the factorization queried before `compute()`

Returns

Constructs the error; `what()` names the factorization and mentions `compute()` .

3 Vector

`Vector<T>` is an owning dense vector, kept distinct from an $n \times 1$ `Matrix` so that `dot` , `outer` , and `norm` have unambiguous meaning. All arithmetic operators are members. Shape mismatches throw `DimensionMismatch` ; `operator==` is exact equality (use `isApprox` for floating-point comparison). `Real` is `NumericTraits<T>::Real` — the magnitude type.

3.1 Construction

`Vector()` / `Vector(Index size)` / `Vector(Index size, const T& fill)`

Example

```
Vector<double> empty;                 assert(empty.isEmpty());
Vector<double> z(4);                  assert(z(0) == 0.0); // value-initialized
Vector<double> filled(4, 7.0);        assert(filled(3) == 7.0);
```

Description

The default constructor makes a length-zero vector; `Vector(size)` makes a zero-initialized vector of the given length; `Vector(size, fill)` sets every element to `fill`.

Parameters

`size` number of elements
`fill` value assigned to every element (fill overload)

Returns

The constructed vector.

`Vector(std::initializer_list<T>)` / `Vector(const std::vector<T>&)`

Example

```
Vector<double> a{1.0, 2.0, 3.0};
assert(a.size() == 3 && a(0) == 1.0 && a(2) == 3.0);
std::vector<double> raw{5.0, 6.0};
Vector<double> fromStd(raw);
assert(fromStd.size() == 2 && fromStd(1) == 6.0);
```

Description

Constructs a vector from a braced list of values, or by copying a `std::vector`. The `std::vector` overload is `explicit`.

Parameters

`values` elements, in order

Returns

The constructed vector.

`Copy / move constructors and assignment`

Example

```
Vector<double> a{1.0, 2.0, 3.0};
Vector<double> copy(a);              assert(copy == a);     // deep copy
Vector<double> moved(std::move(copy)); assert(moved == a);  // steals storage
Vector<double> assigned; assigned = a; assert(assigned == a);
Vector<double> m; m = std::move(assigned); assert(m == a);
```

Description

Copy operations deep-copy the elements; move operations take over the other vector's storage and leave it empty.

Parameters

`other` vector to copy from / move from

Returns

A copy / reference to this vector, per the operation.

3.2 Named constructors

`static Vector Zeros / Ones / Constant / Unit(Index size, ...)`

Example

```
assert(Vector<double>::Zeros(3)    == Vector<double>({0, 0, 0}));
assert(Vector<double>::Ones(2)     == Vector<double>({1, 1}));
assert(Vector<double>::Constant(2, 9.0) == Vector<double>({9, 9}));
assert(Vector<double>::Unit(3, 1)  == Vector<double>({0, 1, 0}));
EXPECT_LINALG_THROW(Vector<double>::Unit(3, 3)); // axis out of range
```

Description

Factory functions for common vectors: all zeros, all ones, a constant fill, and the standard basis vector e_{axis} (1 at `axis`, 0 elsewhere).

Parameters

`size` number of elements
`value` fill value (`Constant`)
`axis` index of the single nonzero entry (`Unit`, 0-based)

Returns

The requested length-`size` vector.

`static Vector Random(Index size, unsigned long seed)`

Example

```
auto r1 = Vector<double>::Random(4, 123);
auto r2 = Vector<double>::Random(4, 123);
assert(r1 == r2);                        // reproducible per seed
for (std::size_t i = 0; i < 4; ++i) assert(r1(i) >= -1.0 && r1(i) <= 1.0);
```

Description

Creates a vector of pseudo-random entries in $[-1, 1]$, deterministic for a given seed so tests are reproducible.

Parameters

`size`	number of elements
`seed`	seed determining the entries

Returns

A length-`size` random vector.

`static Vector LinSpace(Index size, const T& begin, const T& end)`

Example

```
auto ls = Vector<double>::LinSpace(5, 0.0, 1.0);
for (std::size_t i = 0; i < 5; ++i) assert(std::abs(ls(i) - 0.25 * i) < 1e-12);
assert(ls(4) == 1.0);                              // endpoints inclusive
assert(Vector<double>::LinSpace(1, 2.0, 9.0) == Vector<double>({2.0}));
```

Description

Creates a vector of `size` equally spaced values from `begin` to `end`, with both endpoints included.

Parameters

`size`	number of elements
`begin`	first value
`end`	last value (inclusive)

Returns

The linearly spaced vector.

3.3 Element access

`T& operator()(Index i)  /  T& operator[](Index i)  (and const overloads)`

Example

```
Vector<double> v{10.0, 20.0, 30.0};
v(1) = 99.0;
assert(v[1] == 99.0);   // operator[] and operator() are equivalent, unchecked
```

Description

Unchecked element access. `operator()` and `operator[]` are equivalent; each has a mutable and a read-only overload.

Parameters

`i` element index (0-based)

Returns

Reference to element `i`.

`T& at(Index i)  /  const T& at(Index i) const`

Example

```
Vector<double> v{10.0, 20.0, 30.0};
v.at(2) = 42.0; assert(v(2) == 42.0);
try { v.at(5); }
catch (const IndexOutOfRange& e) { assert(e.index() == 5 && e.bound() == 3); }
```

Description

Bounds-checked element access.

Parameters

`i` element index (0-based)

Returns

Reference to element `i`; throws `IndexOutOfRange` if $i \geq \text{size}()$.

`T* data()` / `Index size()` / `bool isEmpty()`

Example

```
Vector<double> v{10.0, 20.0, 30.0};
double* p = v.data();
p[0] = -1.0; assert(v(0) == -1.0);   // contiguous, writes through
assert(v.size() == 3 && !v.isEmpty());
```

Description

`data` exposes the contiguous underlying buffer (mutable and read-only overloads); `size` is the length; `isEmpty` is true when the length is zero.

Returns

The pointer / length / emptiness, respectively.

3.4 Arithmetic

`operator+ / operator- (binary and unary) / operator* / operator/ (scalar)`

Example

```
Vector<double> a{1, 2, 3}, b{4, 5, 6};
assert(a + b == Vector<double>({5, 7, 9}));
assert(b - a == Vector<double>({3, 3, 3}));
assert(a * 2.0 == Vector<double>({2, 4, 6}));
assert(b / 2.0 == Vector<double>({2, 2.5, 3}));
assert(-a == Vector<double>({-1, -2, -3}));
EXPECT_LINALG_THROW(a + Vector<double>({1, 2})); // size mismatch
```

Description

Elementwise addition and subtraction (matching sizes required), scalar multiplication and division, and unary negation. Each returns a new vector.

Parameters

`rhs` vector of matching size (`+`, `-`)
`scalar` scalar multiplier / divisor (`*`, `/`)

Returns

The resulting vector; `+` / `-` throw `DimensionMismatch` on size mismatch.

`operator+= / operator-= / operator*= / operator/=`

Example

```
Vector<double> a{1, 2, 3}, b{4, 5, 6};
Vector<double> c = a;
Vector<double>& ref = (c += b);
assert(&ref == &c && c == Vector<double>({5, 7, 9})); // returns *this
c -= b; assert(c == a);
c *= 10.0; c /= 10.0; assert(c == a);
```

Description

In-place compound assignment mirroring the binary operators; each mutates this vector and returns a reference to it.

Parameters

`rhs` vector of matching size (`+=` , `-=`)
`scalar` scalar multiplier / divisor (`*=` , `/=`)

Returns

Reference to this vector; `+=` / `-=` throw `DimensionMismatch` on mismatch.

`bool operator==(const Vector&) / operator!=(const Vector&)`

Example

```
Vector<double> a{1, 2, 3};
assert(a == Vector<double>({1, 2, 3}));
assert(a != Vector<double>({1, 2}));  // different size
```

Description

Exact equality / inequality: equal means same size and every element exactly equal. For floating-point tolerance use `isApprox` .

Parameters

`rhs` vector to compare against

Returns

`true` / `false` per the comparison.

`Vector scaledBy(const T&) / elementwiseProduct / elementwiseQuotient`

Example

```
Vector<double> a{1, 2, 3}, b{4, 5, 6};
assert(a.scaledBy(3.0) == Vector<double>({3, 6, 9}));
assert(a.elementwiseProduct(b) == Vector<double>({4, 10, 18}));
assert(b.elementwiseQuotient(a) == Vector<double>({4, 2.5, 2}));
```

Description

`scaledBy` is a named mirror of `operator*` (scalar). `elementwiseProduct` and `elementwiseQuotient` are the Hadamard product and quotient.

Parameters

`scalar` scalar multiplier (`scaledBy`)
`rhs` vector of matching size (elementwise ops)

Returns

The resulting vector; elementwise ops throw `DimensionMismatch` on mismatch.

3.5 Products

`T dot(const Vector&)  /  T hermitianDot(const Vector&)`

Example

```
Vector<double> a{1, 2, 3}, b{4, 5, 6};
assert(a.dot(b) == 32.0);          // 4 + 10 + 18
// hermitianDot conjugates the LEFT argument
Vector<C> x{C(1, 2), C(3, -1)}, y{C(2, 0), C(1, 1)};
assert(std::abs(x.hermitianDot(y) - C(4, 0)) < 1e-12);
assert(std::abs(x.hermitianDot(x) - C(15, 0)) < 1e-12); // == ||x||^2
```

Description

`dot` is the unconjugated bilinear form $\sum x_i y_i$, even for complex `T`. `hermitianDot` conjugates **this** (the left argument): $\sum \overline{x_i} y_i$. The two coincide for real scalars; inner-product norms want `hermitianDot`.

Parameters

`rhs` vector of matching size

Returns

The scalar product; throws `DimensionMismatch` on size mismatch.

`Matrix<T> outer(const Vector& rhs)`

Example

```
Vector<double> x{1, 2}, y{10, 20, 30};
Matrix<double> o = x.outer(y);   // 2x3, (i,j) = x_i * y_j
assert(o(0, 0) == 10.0 && o(1, 1) == 40.0);
// complex outer conjugates the right factor: i * conj(i) = 1
Vector<C> cx{C(0, 1)}, cy{C(0, 1)};
assert(cx.outer(cy)(0, 0) == C(1, 0));
```

Description

Outer product xy^H (the right factor is conjugated for complex `T`).

Parameters

`rhs` right operand

Returns

A size() × rhs.size() matrix.

`Vector cross(const Vector& rhs)`

Example

```
Vector<double> e1{1, 0, 0}, e2{0, 1, 0};
assert(e1.cross(e2) == Vector<double>({0, 0, 1}));
assert(e2.cross(e1) == Vector<double>({0, 0, -1})); // anticommutative
EXPECT_LINALG_THROW(e1.cross(Vector<double>({1, 2}))); // non-3 throws
```

Description

Cross product, defined for 3-vectors only.

Parameters

`rhs` right operand; must have size 3

Returns

The cross-product vector; throws `DimensionMismatch` if either vector is not size 3.

`Vector axpy(const T& alpha, const Vector& y)`

Example

```
// fused scale-and-add: alpha * (*this) + y
V lhs = (a + b) * x;
V rhs = (a * x).axpy(1.0, b * x);   // == a*x + b*x
assert(lhs.isApprox(rhs, 1e-10));
```

Description

Fused scale-and-add, $\alpha \cdot (\text{*this}) + y$.

Parameters

`alpha` scalar multiplier applied to this vector
`y` vector added to the scaled result; must match size

Returns

The vector $\alpha \cdot (\text{*this}) + y$; throws `DimensionMismatch` on mismatch.

3.6 Norms & reductions

`Real norm() / squaredNorm() / oneNorm() / infinityNorm() / pNorm(Real p)`

Example

```
Vector<double> v{3.0, -4.0};
assert(std::abs(v.norm() - 5.0) < 1e-12);          // Euclidean
assert(std::abs(v.squaredNorm() - 25.0) < 1e-12);
assert(std::abs(v.oneNorm() - 7.0) < 1e-12);       // sum |x_i|
assert(std::abs(v.infinityNorm() - 4.0) < 1e-12);  // max |x_i|
assert(std::abs(v.pNorm(1.0) - 7.0) < 1e-12);
// norm() scales to avoid overflow; squaredNorm() may overflow
Vector<double> big{1e200, 1e200};
assert(std::isfinite(big.norm()) && std::isinf(big.squaredNorm()));
```

Description

`norm` is the Euclidean (2-)norm, computed with scaling so entries near the overflow threshold do not square to infinity. `squaredNorm` is the unscaled $\sum |x_i|^2$ and may overflow where `norm` does not. `oneNorm`, `infinityNorm`, and `pNorm` are the 1-, ∞-, and general p-norms. An empty vector has norm 0.

Parameters

`p` the norm order, $p \geq 1$ (`pNorm`)

Returns

The requested norm as `Real`.

`T sum() / T product()`

Example

```
Vector<double> a{1, 2, 3};
assert(a.sum() == 6.0);
assert(a.product() == 6.0);
```

Description

Sum and product of all elements.

Returns

The total / product as `T`.

`Vector normalized()` / `void normalize()`

Example

```
Vector<double> v{3.0, 4.0};
Vector<double> u = v.normalized();  assert(std::abs(u.norm() - 1.0) < 1e-12);
v.normalize();                      assert(std::abs(v.norm() - 1.0) < 1e-12);
```

Description

Scales to unit Euclidean norm: `normalized` returns a new vector, `normalize` scales this one in place. Both throw `LinalgError` if the vector is zero.

Returns

The unit vector (`normalized`) / nothing (`normalize`).

`Index maxAbsIndex()` / `minAbsIndex()` / `T maxCoefficient()` / `minCoefficient()`

Example

```
Vector<double> v{-5.0, 2.0, -1.0};
assert(v.maxCoefficient() == 2.0 && v.minCoefficient() == -5.0); // by value
assert(v.maxAbsIndex() == 0 && v.minAbsIndex() == 2);            // by magnitude
EXPECT_LINALG_THROW(Vector<double>().maxCoefficient());          // empty throws
```

Description

`maxAbsIndex` / `minAbsIndex` return the index of the largest / smallest **magnitude**. `maxCoefficient` / `minCoefficient` return the largest / smallest **coefficient** (by real part for complex `T`, since $\mathbb{C}$ has no natural order). Ties resolve to the lowest index; an empty vector throws `LinalgError`.

Returns

The index (`Index`) / coefficient (`T`).

`bool isApprox(const Vector& other, Real tolerance) / bool hasNaN()`

Example

```
Vector<double> a{1, 2, 3}, b{1 + 1e-10, 2 - 1e-10, 3};
assert(a.isApprox(b, 1e-9) && !a.isApprox(b, 1e-12));
assert(!a.isApprox(Vector<double>({1, 2}), 1e9)); // size mismatch -> false
assert(Vector<double>({1, std::nan(""), 3}).hasNaN());
```

Description

`isApprox` tests same-size, per-element agreement within an absolute `tolerance` (a size mismatch is simply `false`, not an exception). `hasNaN` reports whether any element (real or imaginary part) is NaN.

Parameters

`other` vector to compare against (`isApprox`)
`tolerance` absolute per-element tolerance (`isApprox`)

Returns

`true` / `false` per the test.

3.7 Shape manipulation

`Vector segment(Index start, Index count) / head(Index) / tail(Index)`

Example

```
Vector<double> v{1, 2, 3, 4, 5};
assert(v.segment(1, 3) == Vector<double>({2, 3, 4}));
assert(v.head(2) == Vector<double>({1, 2}));
assert(v.tail(2) == Vector<double>({4, 5}));
```

Description

Extract a contiguous sub-vector: `segment` from `start` for `count` elements, `head` the leading `count`, `tail` the trailing `count`.

Parameters

`start` index of the first element (`segment`, 0-based)
`count` number of elements to take

Returns

The requested sub-vector; throws `IndexOutOfRange` if the range exceeds `size()`.

`Vector concat(const Vector& rhs)  /  Vector reversed()`

Example

```
Vector<double> a{1, 2}, b{3, 4};
assert(a.concat(b) == Vector<double>({1, 2, 3, 4}));
assert(a.reversed() == Vector<double>({2, 1}));
```

Description

`concat` appends `rhs` after this vector; `reversed` returns the elements in reverse order.

Parameters

`rhs` vector to append (`concat`)

Returns

The concatenated / reversed vector.

`Matrix<T> asColumnMatrix() / asRowMatrix() / asDiagonalMatrix()`

Example

```
Vector<double> x{1, 2};
assert(x.asColumnMatrix().cols() == 1 && x.asColumnMatrix()(1, 0) == 2.0);
assert(x.asRowMatrix().rows() == 1 && x.asRowMatrix()(0, 1) == 2.0);
Matrix<double> d = x.asDiagonalMatrix();
assert(d(0, 0) == 1.0 && d(1, 1) == 2.0 && d(0, 1) == 0.0);
```

Description

Materialize the vector as a matrix: a size() × 1 column, a 1 × size() row, or a size() × size() diagonal matrix. Each returns an owning copy.

Returns

The requested matrix.

`void resize(Index)  /  void conservativeResize(Index)`

Example

```
Vector<double> r{1, 2, 3};
r.resize(2);              assert(r == Vector<double>({0, 0})); // discards
Vector<double> v{1, 2, 3};
v.conservativeResize(5);  assert(v == Vector<double>({1, 2, 3, 0, 0})); // keeps
```

Description

`resize` changes the length, discarding all existing contents. `conservativeResize` keeps overlapping elements and zero-fills any growth.

Parameters

`size` new length

Returns

Nothing.

`void fill(const T&) / setZero() / setUnit(Index axis) / swap(Vector&)`

Example

```
Vector<double> v(3);
v.fill(5.0);      assert(v(0) == 5.0);
v.setZero();      assert(v(0) == 0.0);
v.setUnit(1);     assert(v == Vector<double>({0, 1, 0}));
Vector<double> a{1, 2}, b{3, 4}; a.swap(b);
assert(a(0) == 3 && b(0) == 1);
```

Description

In-place mutators: set every element to a value / to zero / to the basis vector e_{axis}, or swap contents with another vector.

Parameters

`value` value written to each element (`fill`)
`axis` index of the single nonzero entry (`setUnit` , 0-based)
`other` vector to swap with (`swap`)

Returns

Nothing.

`std::string toString(int precision) const`

Example

```
assert(Vector<double>().toString(3) == "[]");
assert(Vector<double>({1.0}).toString(3) == "[1]");
```

Description

Formats the vector as a text string, for logging.

Parameters

`precision` number of digits shown per element

Returns

A human-readable string.

4 Views

A `MatrixView` is a non-owning, stride-aware window onto another matrix's storage. Element (i, j) lives at `data[i * rowStride + j * colStride]`, so arbitrary strides make rows, columns, diagonals, and transposes all expressible as views without moving any elements. Blocked algorithms operate on views so panels are never copied. A view is invalidated when the owning matrix resizes or dies.

Copy construction and copy assignment **rebind** the view (shallow: same pointer, shape, and strides); they do not touch elements. Assigning from a `Matrix`, or calling `copyFrom`, copies **elements** into the viewed storage and requires matching shapes. `ConstMatrixView` is the read-only counterpart and converts implicitly from `MatrixView`.

4.1 Construction & assignment

`MatrixView()`

Example

```
MatrixView<double> v;
assert(v.rows() == 0 && v.cols() == 0);
assert(v.isEmpty() && v.data() == nullptr);
```

Description

Constructs an empty view: null data, zero shape and strides.

Returns

An empty view.

`MatrixView(T* data, Index rows, Index cols, Index rowStride, Index colStride)`

Example

```
double a[12];                       // a 3x4 row-major buffer
MatrixView<double> v(a, 3, 4, 4, 1);
assert(v.rows() == 3 && v.cols() == 4);
assert(v.rowStride() == 4 && v.colStride() == 1);
// a column-major window over the same kind of buffer uses strides (1, rows)
double c[6] = {1, 2, 3, 4, 5, 6}; // columns (1,2,3) and (4,5,6)
MatrixView<double> cm(c, 3, 2, 1, 3);
assert(cm(2, 0) == 3 && cm(0, 1) == 4);
```

Description

Constructs a view over caller-owned storage. `data` points at the view's element $(0, 0)$; `rowStride` / `colStride` are how many elements to advance to move down one row / right one column.

Parameters

`data` pointer to the element at position $(0, 0)$ of the view
`rows` number of rows the view exposes
`cols` number of columns the view exposes
`rowStride` elements to advance in `data` to move down one row
`colStride` elements to advance in `data` to move right one column

Returns

A view over the given storage.

`MatrixView(const MatrixView& other) / MatrixView(MatrixView&& other)`

Example

```
auto v = /* a 3x4 view over buffer b */;
MatrixView<double> w(v);        // shallow: same storage
assert(w.data() == v.data());
w(0, 0) = 42.0;
assert(v(0, 0) == 42.0);        // writes visible through the original
MatrixView<double> m(std::move(v));
assert(v.isEmpty() && v.data() == nullptr); // moved-from is empty
```

Description

Copy construction rebinds this view to the same storage as `other` (shallow: shares pointer, shape, strides). Move construction takes over `other`'s binding and leaves it empty; since the view is non-owning it is equivalent to a shallow copy.

Parameters

`other` view to copy the binding from / move from

Returns

A view bound to `other`'s storage.

`MatrixView& operator=(const MatrixView&) / operator=(MatrixView&&)`

Example

```
double other[4] = {7, 7, 7, 7};
MatrixView<double> o(other, 2, 2, 2, 1);
o = v;                      // rebinds; does NOT copy elements
assert(o.data() == v.data() && o.rows() == 3);
assert(other[0] == 7.0);    // the old target storage is untouched
```

Description

Rebinds this view to `other`'s storage (shallow); no elements are copied. Move assignment additionally empties `other` and is safe under self-assignment.

Parameters

`other` view to rebind to / move from

Returns

Reference to this view.

`MatrixView& operator=(const Matrix<T>& source)`

Example

```
auto blk = a.block(0, 0, 2, 2);   // view into matrix a
blk = Matrix<double>{{-1, -2}, {-4, -5}};
assert(a(0, 0) == -1 && a(1, 1) == -5); // elements copied into a
assert(a(0, 2) == 3);                   // outside the block untouched
```

Description

Copies the elements of `source` into the storage this view refers to; the binding is untouched. Shapes must match exactly.

Parameters

`source` matrix whose element values are copied in

Returns

Reference to this view.

4.2 Element access & shape

`T& operator()(Index i, Index j)  /  const T& operator()(...) const`

Example

```
assert(v(0, 0) == 0.0 && v(1, 2) == 12.0 && v(2, 3) == 23.0);
v(1, 2) = -5.0;
assert(b.a[6] == -5.0); // writes land in the underlying buffer
```

Description

Unchecked element access, honoring the strides. Indices are relative to the view, not to the owning matrix.

Parameters

`i` row index, relative to this view (0-based)
`j` column index, relative to this view (0-based)

Returns

Reference to element (i, j) in the viewed storage.

`T& at(Index i, Index j)  /  const T& at(...) const`

Example

```
assert(v.at(2, 3) == 23.0);
v.at(0, 1) = 99.0;
bool threw = false;
try { v.at(3, 0); } catch (const IndexOutOfRange&) { threw = true; }
assert(threw);
```

Description

Bounds-checked element access. Otherwise identical to `operator()`.

Parameters

- `i` row index, relative to this view (0-based)
- `j` column index, relative to this view (0-based)

Returns

Reference to element (i, j); throws `IndexOutOfRange` if $i \geq \text{rows}()$ or $j \geq \text{cols}()$.

`Index rows() / cols() / rowStride() / colStride() const`

Example

```
MatrixView<double> v(a, 3, 4, 4, 1);
assert(v.rows() == 3 && v.cols() == 4);
assert(v.rowStride() == 4 && v.colStride() == 1);
```

Description

The view's shape and the strides that map indices to storage offsets.

Returns

The requested count as `Index`.

`bool isContiguous() / isEmpty() const`

Example

```
assert(b.view().isContiguous());              // dense, row-major, gap-free
assert(!b.view().block(0, 0, 3, 2).isContiguous()); // skips columns
assert(b.view().block(1, 0, 2, 4).isContiguous());  // full-width block
assert(MatrixView<double>(&x, 0, 5, 5, 1).isEmpty());
```

Description

`isContiguous` reports whether the view is a dense row-major block with no gaps (rowStride = cols and colStride = 1), so the storage can be treated as a flat buffer. `isEmpty` reports whether `rows()` or `cols()` is zero.

Returns

`true` if the view is contiguous / empty, otherwise `false`.

`T* data()  /  const T* data() const`

Example

```
assert(v.data() == b.a); // points at element (0, 0)
// honor the strides unless isContiguous() is true
```

Description

Direct access to the underlying buffer at position $(0, 0)$. Callers must honor the strides unless `isContiguous()` is true.

Returns

Pointer to the element at position $(0, 0)$.

4.3 Sub-views

`MatrixView block(Index i, Index j, Index numRows, Index numCols)`

Example

```
auto blk = v.block(1, 1, 2, 3);
assert(blk.rows() == 2 && blk.cols() == 3);
assert(blk(0, 0) == 11.0 && blk(1, 2) == 23.0);
blk(0, 1) = -1.0;              // writes visible in the parent
auto inner = blk.block(1, 1, 1, 2); // nested indices are block-relative
assert(inner(0, 0) == 22.0);
```

Description

Creates a sub-view over a rectangular block, sharing this view's storage. Indices are relative to this view, and nested blocks compose relative to their parent.

Parameters

- **`i`** row index of the block's top-left corner (0-based)
- **`j`** column index of the block's top-left corner (0-based)
- **`numRows`** number of rows in the block
- **`numCols`** number of columns in the block

Returns

A view onto the block; throws `IndexOutOfRange` if the block does not fit within this view.

`MatrixView row(Index i)  /  MatrixView col(Index j)`

Example

```
auto r = v.row(1);  assert(r.rows() == 1 && r.cols() == 4);
auto c = v.col(2);  assert(c.rows() == 3 && c.cols() == 1);
c(1, 0) = 77.0;     // shares storage
assert(v(1, 2) == 77.0);
```

Description

Creates a $1 \times \text{cols}$ view onto a single row, or a $\text{rows} \times 1$ view onto a single column, sharing this view's storage.

Parameters

`i / j` row index / column index (0-based)

Returns

A one-row / one-column view; throws `IndexOutOfRange` if the index is out of range.

`MatrixView diagonal()`

Example

```
auto d = v.diagonal(); // 1 x min(rows, cols)
assert(d.rows() == 1 && d.cols() == 3);
assert(d(0, 0) == 0.0 && d(0, 1) == 11.0 && d(0, 2) == 22.0);
d(0, 1) = -3.0;
assert(v(1, 1) == -3.0);
```

Description

Creates a $1 \times \min(\text{rows}, \text{cols})$ view onto the main diagonal, sharing this view's storage.

Returns

A one-row view onto the diagonal.

`MatrixView transposed()`

Example

```
auto t = v.transposed();
assert(t.rows() == 4 && t.cols() == 3);
assert(t.rowStride() == v.colStride() && t.colStride() == v.rowStride());
t(3, 0) = -9.0;             // shares storage
assert(v(0, 3) == -9.0);
```

Description

Creates a transposed view by swapping shape and strides; no elements are moved. The round-trip `transposed().transposed()` recovers the original.

Returns

A cols × rows view onto the same storage.

`Matrix<T> toMatrix() const`

Example

```
M blockCopy = a.block(0, 1, 2, 2).toMatrix(); // deep copy
blockCopy(0, 0) = 99;
assert(a(0, 1) == 2);   // source unaffected
M t = a.view().transposed().toMatrix(); // works through strided views
```

Description

Deep-copies the viewed elements into a fresh contiguous `Matrix`, walking the strides so transposed and other strided views copy correctly.

Returns

A newly allocated `Matrix` holding a copy of the elements.

4.4 Element mutation

`void fill(const T& value)  /  void setZero()`

Example

```
auto blk = v.block(0, 0, 2, 2);
blk.fill(5.0);
assert(v(0, 0) == 5.0 && v(1, 1) == 5.0);
assert(v(0, 2) == 2.0); // outside the block untouched
blk.setZero();
assert(v(0, 0) == 0.0);
```

Description

Writes a constant (or zero) to every element the view refers to; storage outside the view is left alone.

Parameters

`value` value written to each element (`fill` only)

Returns

Nothing.

`void scale(const T& factor)`

Example

```
blk.scale(3.0);
assert(v(0, 0) == 15.0); // 5.0 * 3.0
// complex: scale by i rotates each element
MatrixView<C> cv(a, 2, 2, 2, 1);
cv.scale(C(0, 1));
```

Description

Multiplies every element of the view in place by `factor`.

Parameters

`factor` scalar multiplier applied to each element

Returns

Nothing.

`void swapWith(MatrixView& other)`

Example

```
MatrixView<double> va(a, 2, 2, 2, 1), vb(b, 2, 2, 2, 1);
va.swapWith(vb);
assert(a[0] == 5 && b[0] == 1); // elements exchanged...
assert(va.data() == a && vb.data() == b); // ...bindings stay put
```

Description

Exchanges the elements of this view with those of `other`; the bindings are left unchanged. Shapes must match.

Parameters

`other` view whose elements are exchanged with this view's

Returns

Nothing.

`void copyFrom(const MatrixView& source)`

Example

```
vd.copyFrom(vs);
assert(dst[0] == 1 && dst[3] == 4);
assert(src[0] == 1);            // source untouched
vd2.copyFrom(vs.transposed()); // copies across differing strides
```

Description

Copies elements from `source` into this view. Shapes must match and the source must not alias this view's storage; differing strides (e.g. a transposed source) are handled correctly.

Parameters

`source` view to copy element values from

Returns

Nothing.

4.5 ConstMatrixView

`ConstMatrixView` mirrors the read-only half of the API — construction, element access (`operator()`, `at`), `rows` / `cols` / `rowStride` / `colStride`, `isContiguous` / `isEmpty`, `data() const`, the sub-view factories (`block`, `row`, `col`, `diagonal`, `transposed`), and `toMatrix` — with identical semantics. It adds one constructor for the mutable-to-const conversion.

`ConstMatrixView(const MatrixView<T>& view)`

Example

```
auto mv = b.view();                       // mutable view
ConstMatrixView<double> converted = mv;  // implicit conversion
assert(converted.data() == b.a && converted.rows() == 3);
// sub-views mirror the mutable API
auto blk = converted.block(1, 1, 2, 3);
assert(blk(0, 0) == 11.0 && blk(1, 2) == 23.0);
```

Description

Implicitly constructs a read-only view from a mutable one, so a `MatrixView` can be passed wherever only reads are needed. (There is also the direct `ConstMatrixView(const T* data, rows, cols, rowStride, colStride)` constructor, mirroring the mutable one over `const` storage.)

Parameters

`view` mutable view to expose as read-only

Returns

A read-only view sharing `view`'s storage.

5 Matrix

`Matrix<T>` is an owning dense matrix, row-major and contiguous: element (i, j) lives at `data()[i * cols() + j]`. Every operator is a member, so `matrix * scalar` exists but `scalar * matrix` does not — use `scaledBy` for the reversed form. Shape mismatches throw `DimensionMismatch`; `operator==` is exact equality (use `isApprox` for floating-point comparison). Several scalar summaries (`determinant`, `spectralNorm`, `conditionNumber`, `rank`, `inverse`, `pseudoInverse`) factorize internally and are $O(n^3)$ conveniences — hold a decomposition object when you need more than one query.

5.1 Construction

`Matrix()  /  Matrix(Index rows, Index cols)  /  Matrix(rows, cols, const T& fill)`

Example

```
M e;              assert(e.rows() == 0 && e.isEmpty());
M a(2, 3);        assert(a.size() == 6 && a(0, 0) == 0.0); // value-initialized
M f(2, 2, 7.5);   assert(f(0, 0) == 7.5 && f(1, 1) == 7.5);
```

Description

The default constructor makes an empty 0×0 matrix; `Matrix(rows, cols)` makes a zero-initialized matrix of the given shape; the third overload fills every element with `fill`.

Parameters

`rows`	number of rows
`cols`	number of columns
`fill`	value assigned to every element (fill overload)

Returns

The constructed matrix.

`Matrix(rows, cols, const std::vector<T>&)` / `Matrix(nested initializer_list)`

Example

```
M b(2, 3, std::vector<double>{1, 2, 3, 4, 5, 6}); // flat, row-major
assert(b(1, 0) == 4 && b(1, 2) == 6);
M a{{1, 2, 3}, {4, 5, 6}};                        // braced rows
assert(a.rows() == 2 && a(1, 2) == 6);
```

Description

Build from a flat row-major element list (size must equal rows × cols), or from a braced list of rows (every inner list must have equal length).

Parameters

`rows / cols` shape (flat overload)
`rowMajorData` elements in row-major order

Returns

The constructed matrix.

Copy / move / from-view constructors and assignment

Example

```
M a{{1, 2}, {3, 4}};
M b(a); b(0, 0) = 99; assert(a(0, 0) == 1 && b(0, 0) == 99); // deep copy
M c(std::move(b));    assert(c(0, 0) == 99 && b.isEmpty());  // steals storage
M fromView(someConstView);                                   // explicit
```

Description

Copy operations deep-copy the elements; move operations take over the other matrix's storage and leave it empty. `explicit Matrix(const ConstMatrixView<T>&)` deep-copies a view's elements into a fresh contiguous matrix.

Parameters

`other` matrix to copy from / move from
`view` read-only view to copy (from-view constructor)

Returns

A copy / reference to this matrix, per the operation.

5.2 Named constructors

`static Matrix Zeros / Ones / Constant / Identity / Diagonal`

Example

```
assert(M::Zeros(2, 3)(1, 2) == 0.0);
assert(M::Ones(2, 2)(0, 1) == 1.0);
assert(M::Constant(2, 2, 3.5)(1, 0) == 3.5);
M i = M::Identity(3);       assert(i(0, 0) == 1 && i(0, 1) == 0);
M d = M::Diagonal(V{5, 7}); assert(d(0, 0) == 5 && d(1, 1) == 7 && d(0, 1) == 0);
```

Description

Factories for common matrices: all zeros, all ones, a constant fill, the square identity, and a square diagonal matrix built from a vector.

Parameters

`rows / cols` shape (`Zeros` , `Ones` , `Constant`)
`size` side length (`Identity`)
`value` fill value (`Constant`)
`values` diagonal entries (`Diagonal`)

Returns

The requested matrix.

`static Matrix Random / RandomSymmetric / RandomOrthogonal(size, seed)`

Example

```
M a = M::Random(3, 4, 42), b = M::Random(3, 4, 42);
assert(a == b);                          // same seed -> same entries
assert(M::RandomSymmetric(4, 1).isSymmetric(0.0));
assert(M::RandomOrthogonal(4, 7).isOrthogonal(1e-12));
```

Description

Deterministic random matrices (reproducible per seed). `Random` fills all entries; `RandomSymmetric` is symmetric (Hermitian for complex `T`); `RandomOrthogonal` is orthogonal/unitary — a perfectly conditioned Q factor.

Parameters

`rows / cols` shape (`Random`)
`size` side length (`RandomSymmetric`, `RandomOrthogonal`)
`seed` seed determining the entries

Returns

The requested random matrix.

static Matrix Hilbert(Index size) / Vandermonde(const Vector<T>& nodes, Index degree)

Example

```
M h = M::Hilbert(3);              // a(i,j) = 1/(i+j+1); ill-conditioned
assert(std::abs(h(0, 1) - 0.5) < 1e-12 && h.isSymmetric(0.0));
M v = M::Vandermonde(V{1, 2, 3}, 2); // a(i,j) = nodes[i]^j
assert(v.cols() == 3 && v(1, 2) == 4 && v(2, 2) == 9);
```

Description

Two structured test inputs: the Hilbert matrix $a(i, j) = \frac{1}{i+j+1}$, the classic ill-conditioned stress case, and the Vandermonde matrix $a(i, j) = \text{nodes}[i]^j$ for j from 0 to `degree`.

Parameters

`size` side length (`Hilbert`)
`nodes` node values, one per row (`Vandermonde`)
`degree` highest power; the result has degree + 1 columns

Returns

The requested matrix.

static Matrix FromColumns / FromRows(const std::vector<Vector<T>>&)

Example

```
M a = M::FromColumns({V{1, 2}, V{3, 4}});
assert(a(0, 0) == 1 && a(1, 0) == 2 && a(0, 1) == 3);
M b = M::FromRows({V{1, 2}, V{3, 4}});
assert(b(0, 0) == 1 && b(0, 1) == 2 && b(1, 0) == 3);
```

Description

Assemble a matrix from column vectors (left to right) or row vectors (top to bottom); all vectors must have equal length.

Parameters

`columns / rows` equal-length vectors forming the columns / rows

Returns

A matrix whose columns / rows are the given vectors.

5.3 Element access & shape

`T& operator()(Index i, Index j)  /  T& at(Index i, Index j)  (+ const overloads)`

Example

```
M a{{1, 2}, {3, 4}};
a.at(0, 1) = 9; assert(a(0, 1) == 9);   // at() is bounds-checked
try { a.at(2, 0); } catch (const IndexOutOfRange&) { /* ... */ }
```

Description

`operator()` is unchecked element access; `at` throws `IndexOutOfRange` when $i \geq$ rows() or $j \geq$ cols(). Each has a mutable and a read-only overload.

Parameters

- `i` row index (0-based)
- `j` column index (0-based)

Returns

Reference to element (i, j).

`T* data()  /  Index rows() / cols() / size()  /  bool isEmpty() / isSquare()`

Example

```
M b(2, 3, std::vector<double>{1, 2, 3, 4, 5, 6});
assert(b.data()[1 * 3 + 2] == 6); // row-major: (i,j) at data()[i*cols + j]
assert(b.rows() == 2 && b.cols() == 3 && b.size() == 6);
assert(!b.isEmpty() && !b.isSquare());
```

Description

`data` exposes the row-major buffer. `rows` / `cols` / `size` report the shape and element count; `isEmpty` is true when either dimension is zero; `isSquare` is true when rows() = cols().

Returns

The pointer / count / predicate, respectively.

5.4 Views

`MatrixView view() / block(...) / rowView(Index) / colView(Index) (+ const)`

Example

```
M a{{0, 1, 2}, {10, 11, 12}, {20, 21, 22}};
auto v = a.view(); v(0, 0) = -5; assert(a(0, 0) == -5); // aliases storage
auto blk = a.block(1, 1, 2, 2); blk.fill(0.0);
assert(a(1, 1) == 0 && a(0, 1) == 1);                   // outside untouched
assert(a.rowView(0).cols() == 3 && a.colView(2).rows() == 3);
```

Description

Return `MatrixView` / `ConstMatrixView` windows that **alias** this matrix's storage: writes through them are visible here, and they dangle after a resize or destruction. `view` covers the whole matrix, `block` a rectangle, `rowView` / `colView` a single row / column.

Parameters

`i, j, numRows, numCols` block position and shape (`block`)
`i / j` row / column index (`rowView` / `colView`)

Returns

A view aliasing this matrix's storage.

`Vector<T> row(Index i) / col(Index j) / diagonal()`

Example

```
M a{{1, 2}, {3, 4}};
V r = a.row(1); r(0) = 99; assert(a(1, 0) == 3); // owning COPY, not a view
assert(a.col(0)(1) == 3);
assert(a.diagonal() == V{1, 4});
```

Description

Copy a row, column, or the main diagonal into an owning `Vector`. Unlike the `*View` members these return independent copies, so mutating the result does not touch the matrix.

Parameters

`i / j` row / column index (0-based)

Returns

A vector holding the copied entries.

`void setRow(Index, const Vector&) / setCol(Index, const Vector&) / setBlock(i, j, const Matrix&)`

Example

```
M a = M::Zeros(3, 3);
a.setRow(0, V{1, 2, 3}); assert(a(0, 2) == 3);
a.setCol(2, V{7, 8, 9}); assert(a(2, 2) == 9);
a.setBlock(1, 0, M{{5, 6}}); assert(a(1, 0) == 5 && a(1, 1) == 6);
```

Description

Overwrite a row (length must equal `cols()`), a column (length must equal `rows()`), or a rectangular block (the `source` matrix defines the block shape, placed with its top-left corner at (i, j)).

Parameters

`i / j` target row / column, or block corner (0-based)
`values` replacement values (`setRow` / `setCol`)
`source` matrix copied into the block (`setBlock`)

Returns

Nothing.

5.5 Arithmetic

`Matrix operator+ / operator- (binary and unary) / operator* (Matrix)`

Example

```
M a{{1, 2, 3}, {4, 5, 6}};      // 2x3
M b{{7, 8}, {9, 10}, {11, 12}}; // 3x2
M p = a * b;                    // 2x2
assert(p(0, 0) == 58 && p(1, 1) == 154);
assert((a * M::Identity(3)).isApprox(a, 1e-10)); // identity is neutral
EXPECT_LINALG_THROW(a * a);     // inner dims 3 vs 2
```

Description

Elementwise addition and subtraction (matching shapes required), unary negation, and matrix-matrix multiplication (the right operand's `rows()` must equal this matrix's `cols()`).

Parameters

`rhs` right operand

Returns

The resulting matrix; throws `DimensionMismatch` on incompatible shapes.

`Vector<T> operator*(const Vector<T>& rhs)`

Example

```
M a{{1, 2}, {3, 4}};
V x{1, -1};
V ax = a * x;              // length rows()
assert(ax(0) == -1 && ax(1) == -1);
```

Description

Matrix-vector multiplication; `rhs.size()` must equal this matrix's `cols()`.

Parameters

`rhs` right operand; its size must equal `cols()`

Returns

The product vector of length `rows()`; throws `DimensionMismatch` on mismatch.

`Matrix operator* / operator/ (scalar)  /  scaledBy(const T&)`

Example

```
M a{{1, 2}, {3, 4}};
assert((a * 2.0)(1, 1) == 8);
assert((a / 2.0)(0, 0) == 0.5);
assert(a.scaledBy(3.0)(0, 1) == 6); // the scalar * A form
```

Description

Scalar multiplication (matrix on the left) and division. `scaledBy` is the named form for scalar-on-the-left, which `operator*` cannot express as a member.

Parameters

`scalar` scalar multiplier / divisor

Returns

The scaled matrix.

`operator+= / operator-= / operator*= (Matrix and scalar) / operator/=`

Example

```
M a{{1, 2}, {3, 4}}, b{{10, 20}, {30, 40}};
a += b; assert(a(1, 0) == 33);
a -= b; assert(a(1, 0) == 3);
M sq{{1, 1}, {0, 1}}, sq2 = sq; sq2 *= sq;
assert(sq2.isApprox(sq * sq, 1e-10));
a *= 10.0; a /= 10.0;
```

Description

In-place compound assignment mirroring the binary operators (matrix add/subtract/multiply, and scalar multiply/divide); each returns a reference to this matrix.

Parameters

`rhs / scalar` right operand or scalar

Returns

Reference to this matrix; matrix forms throw `DimensionMismatch` on mismatch.

`bool operator==(const Matrix&) / operator!=(const Matrix&)`

Example

```
M a{{1, 2}, {3, 4}}, b = a;
assert(a == b);
b(0, 0) += 1e-15;
assert(a != b);                        // exact comparison
assert(!(a == M::Zeros(2, 3)));        // shape mismatch is inequality, not throw
```

Description

Exact equality / inequality: equal means same shape and every element exactly equal. A shape mismatch is simply inequality (never an exception). Use `isApprox` for floating-point tolerance.

Parameters

`rhs` matrix to compare against

Returns

`true` / `false` per the comparison.

Matrix elementwiseProduct / elementwiseQuotient / kroneckerProduct / power

Example

```
M a{{1, 2}, {3, 4}}, b{{2, 2}, {2, 2}};
assert(a.elementwiseProduct(b)(1, 1) == 8);
assert(a.elementwiseQuotient(b)(1, 0) == 1.5);
M k = M{{1, 2}}.kroneckerProduct(M{{0, 1}, {1, 0}}); // 2x4
assert(k(0, 1) == 1 && k(0, 3) == 2);
assert(a.power(0).isApprox(M::Identity(2), 1e-10) && a.power(3).isApprox(a*a*a,
1e-10));
```

Description

`elementwiseProduct` / `elementwiseQuotient` are the Hadamard product and quotient (matching shapes). `kroneckerProduct` is the tensor product, of shape $(\text{rows} \cdot \text{rhs.rows}) \times (\text{cols} \cdot \text{rhs.cols})$. `power` raises a square matrix to a non-negative integer power (exponent 0 yields the identity).

Parameters

`rhs` right operand (elementwise / Kronecker)
`exponent` non-negative power (`power`)

Returns

The resulting matrix; throws `DimensionMismatch` on shape mismatch (or if `power` is called on a non-square matrix).

5.6 Shape manipulation

`Matrix transpose()` / `conjugateTranspose()` / `void transposeInPlace()`

Example

```
M a{{1, 2, 3}, {4, 5, 6}};
M t = a.transpose(); assert(t.rows() == 3 && t(2, 0) == 3 && t.transpose() == a);
MC c{{C(1, 2), C(3, -1)}}; // conjugateTranspose flips the sign of imag parts
assert(c.conjugateTranspose()(0, 0) == C(1, -2));
M sq{{1, 2}, {3, 4}}; sq.transposeInPlace(); assert(sq(0, 1) == 3);
```

Description

`transpose` returns a new transposed matrix; `conjugateTranspose` returns the Hermitian transpose (equal to `transpose` for real `T`); `transposeInPlace` transposes this matrix in place.

Returns

The transposed matrix (`transpose` / `conjugateTranspose`) or nothing (`transposeInPlace`).

`Matrix reshaped(Index rows, Index cols)`

Example

```
M a{{1, 2, 3}, {4, 5, 6}};
M r = a.reshaped(3, 2);   // row-major order preserved
assert(r(0, 0) == 1 && r(1, 0) == 3 && r(2, 1) == 6);
EXPECT_LINALG_THROW(a.reshaped(4, 2)); // 8 != 6
```

Description

Reinterprets the elements (in row-major order) with a new shape; rows × cols must equal `size()`.

Parameters

`rows` new row count
`cols` new column count

Returns

A matrix with the new shape and the same elements; throws `DimensionMismatch` if the element count changes.

`Matrix horizontalConcat / verticalConcat / withoutRow / withoutCol`

Example

```
M h = M{{1}, {3}}.horizontalConcat(M{{2}, {4}}); // 2x2
assert(h(0, 1) == 2 && h(1, 0) == 3);
M v = M{{1, 2}}.verticalConcat(M{{3, 4}});       // 2x2
assert(v(1, 0) == 3);
M a{{1, 2, 3}, {4, 5, 6}};
assert(a.withoutRow(0).rows() == 1 && a.withoutCol(1)(0, 1) == 3);
```

Description

`horizontalConcat` stacks `rhs` to the right (matching row counts); `verticalConcat` stacks it below (matching column counts). `withoutRow` / `withoutCol` return a copy with one row / column removed.

Parameters

`rhs` matrix to stack (concat)
`i / j` row / column index to drop (`withoutRow` / `withoutCol`)

Returns

The reshaped matrix; concat throws `DimensionMismatch` on mismatch, `without*` throws `IndexOutOfRange` if the index is out of range.

`void resize / conservativeResize / swapRows / swapCols / fill / setZero / setIdentity / swap`

Example

```
M a{{1, 2}, {3, 4}};
a.conservativeResize(3, 3); assert(a(1, 1) == 4 && a(2, 2) == 0); // keeps top-left
a.resize(2, 2);             // discards contents
a.setIdentity(); assert(a(0, 0) == 1 && a(0, 1) == 0);
M c{{1, 2}, {3, 4}}; c.swapRows(0, 1); assert(c(0, 0) == 3);
c.swapCols(0, 1); assert(c(0, 0) == 4);
```

Description

In-place shape/content mutators: `resize` (discards contents), `conservativeResize` (keeps the overlapping top-left block, zero-fills growth), `swapRows` / `swapCols`, `fill` / `setZero`, `setIdentity` (square only), and `swap` (exchange contents with another matrix).

Parameters

`rows / cols` new shape (`resize`, `conservativeResize`)
`a, b` indices to swap (`swapRows`, `swapCols`)
`value` fill value (`fill`)
`other` matrix to swap with (`swap`)

Returns

Nothing.

5.7 Scalar summaries

`T trace() / sum() / determinant()`

Example

```
M a{{1, -2}, {3, 4}};
assert(a.trace() == 5.0 && a.sum() == 6.0);
assert(std::abs(M{{1, 2}, {3, 4}}.determinant() - (-2.0)) < 1e-10);
assert(std::abs(M{{1, 2}, {2, 4}}.determinant()) < 1e-10); // singular
```

Description

`trace` sums the main diagonal; `sum` sums all elements; `determinant` computes the determinant of a square matrix via LU factorization.

Returns

The scalar result as `T`; `determinant` throws `DimensionMismatch` if the matrix is not square.

`Real oneNorm() / infinityNorm() / frobeniusNorm() / maxNorm()`

Example

```
M a{{1, -2}, {3, 4}};
assert(a.oneNorm() == 6.0);       // max abs column sum: |-2|+|4|
assert(a.infinityNorm() == 7.0);  // max abs row sum: |3|+|4|
assert(std::abs(a.frobeniusNorm() - std::sqrt(30.0)) < 1e-10);
assert(a.maxNorm() == 4.0);
// complex uses magnitudes: |3+4i| = 5
assert(std::abs(MC{{C(3, 4)}}.frobeniusNorm() - 5.0) < 1e-10);
```

Description

Matrix norms: `oneNorm` (maximum absolute column sum), `infinityNorm` (maximum absolute row sum), `frobeniusNorm` (root of the sum of squared magnitudes), and `maxNorm` (largest element magnitude).

Returns

The requested norm as `Real`.

`Real spectralNorm() / conditionNumber() / Index rank(Real tolerance)`

Example

```
M d = M::Diagonal(V{3, -5});
assert(std::abs(d.spectralNorm() - 5.0) < 1e-10);          // largest sigma
assert(std::abs(d.conditionNumber() - 5.0 / 3.0) < 1e-10); // sigma_max/sigma_min
assert(M{{1, 0}, {0, 2}}.rank(1e-12) == 2);
assert(M{{1, 2}, {2, 4}}.rank(1e-12) == 1);                // rank-deficient
```

Description

SVD-based summaries: `spectralNorm` is the largest singular value; `conditionNumber` is $\frac{\sigma_{\max}}{\sigma_{\min}}$; `rank` counts singular values above tolerance $\cdot\, \sigma_{\max}$. Each runs a full SVD, so prefer a held decomposition for repeated queries.

Parameters

`tolerance` relative singular-value cutoff (`rank`)

Returns

The norm / condition number (`Real`) or the numerical rank (`Index`).

5.8 Predicates

`bool isSymmetric / isHermitian / isDiagonal(Real tolerance)`

Example

```
assert(M{{1, 2}, {2, 3}}.isSymmetric(0.0));
assert(!M{{1, 2}, {3, 4}}.isSymmetric(0.0));
MC h{{C(1, 0), C(2, 3)}, {C(2, -3), C(5, 0)}};
assert(h.isHermitian(0.0));
assert(M::Diagonal(V{1, 2}).isDiagonal(0.0));
```

Description

Structural tests within an absolute per-element tolerance (pass 0 for exact): `isSymmetric` ($A = A^T$), `isHermitian` ($A = A^H$, the meaningful symmetry test for complex `T`), and `isDiagonal` (off-diagonal entries negligible).

Parameters

`tolerance` absolute per-element tolerance; 0 for exact

Returns

`true` if the property holds within tolerance, otherwise `false`.

`bool isTriangular(Triangle::Kind which, Real tolerance) / isOrthogonal(Real tolerance)`

Example

```
M up{{1, 2}, {0, 3}};
assert(up.isTriangular(Triangle::Kind::Upper, 0.0));
assert(!up.isTriangular(Triangle::Kind::Lower, 0.0));
M rot{{0, -1}, {1, 0}};  // 90-degree rotation
assert(rot.isOrthogonal(1e-14));
assert(!M{{2, 0}, {0, 1}}.isOrthogonal(1e-6));
```

Description

`isTriangular` tests whether the matrix is triangular of the given kind (the other triangle negligible); `isOrthogonal` tests $A^H A = I$. Both use an absolute tolerance (0 for exact).

Parameters

`which` which triangle is expected nonzero (`isTriangular`)
`tolerance` absolute per-element tolerance; 0 for exact

Returns

`true` if the property holds within tolerance, otherwise `false`.

`bool isApprox(const Matrix& other, Real tolerance) / bool hasNaN()`

Example

```
M a{{1, 2}, {3, 4}}, b = a; b(0, 0) += 1e-15;
assert(a != b && a.isApprox(b, 1e-12));
M nan{{1, 2}, {3, 4}}; nan(0, 1) = std::nan("");
assert(nan.hasNaN());
```

Description

`isApprox` tests same-shape, per-element agreement within an absolute `tolerance`; `hasNaN` reports whether any element is NaN.

Parameters

`other` matrix to compare against (`isApprox`)
`tolerance` absolute per-element tolerance (`isApprox`)

Returns

`true` / `false` per the test.

5.9 Derived matrices

`Matrix inverse()  /  Matrix pseudoInverse(Real tolerance)`

Example

```
M a{{4, 7}, {2, 6}};
assert((a * a.inverse()).isApprox(M::Identity(2), 1e-10));
EXPECT_LINALG_THROW(M{{1, 2}, {2, 4}}.inverse()); // singular
M tall{{1, 0}, {0, 2}, {0, 0}};
M p = tall.pseudoInverse(1e-12);        // 2x3 left inverse
assert((p * tall).isApprox(M::Identity(2), 1e-10));
```

Description

`inverse` is the matrix inverse via LU (square only). `pseudoInverse` is the Moore-Penrose pseudoinverse via SVD, with `tolerance` the relative singular-value cutoff (as in `rank`).

Parameters

`tolerance` relative singular-value cutoff (`pseudoInverse`)

Returns

The inverse / pseudoinverse; `inverse` throws `SingularMatrix` if singular and `DimensionMismatch` if not square.

`Matrix triangularPart(Triangle::Kind) / symmetricPart() / skewSymmetricPart()`

Example

```
M a{{1, 2}, {3, 4}};
M up = a.triangularPart(Triangle::Kind::Upper);
assert(up(0, 1) == 2 && up(1, 0) == 0 && up(1, 1) == 4);
M sym = a.symmetricPart(), skew = a.skewSymmetricPart();
assert((sym + skew).isApprox(a, 1e-10)); // decomposition identity
```

Description

`triangularPart` keeps one triangle (and the diagonal), zero-filling the other. `symmetricPart` is $(A + A^H)/2$ and `skewSymmetricPart` is $(A - A^H)/2$; the two sum back to A.

Parameters

`which` which triangle to keep (`triangularPart`)

Returns

The requested derived matrix.

5.10 Serialization

`std::string toString(int precision) / std::string toMatlabLiteral()`

Example

```
M a{{1.5, 2}, {3, 4}};
std::string s = a.toString(3);
assert(s.find("1.5") != std::string::npos);
std::string ml = a.toMatlabLiteral(); // e.g. "[a b; c d]"
assert(ml.front() == '[' && ml.back() == ']' && ml.find(';') != std::string::npos);
```

Description

`toString` formats the matrix as aligned rows of text for logging; `toMatlabLiteral` formats it as a MATLAB/Octave literal (rows separated by `;`).

Parameters

`precision` digits shown per element (`toString`)

Returns

The formatted string.

`static Matrix FromCsv(const std::string& path) / void writeCsv(const std::string& path)`

Example

```
M a{{1.25, -2}, {3, 4.5}};
a.writeCsv("m.csv");
M b = M::FromCsv("m.csv");
assert(b.isApprox(a, 1e-12)); // round-trips
```

Description

CSV I/O: `FromCsv` parses a matrix from a file, `writeCsv` writes one. Both throw `LinalgError` on I/O or parse failure.

Parameters

`path` filesystem path to read / write

Returns

The parsed matrix (`FromCsv`) or nothing (`writeCsv`).

6 Givens rotations

A `Givens<T>` is a plane rotation that zeros a single entry — preferred over a Householder reflector where the pattern is sparse (bidiagonal sweeps, QR updating, Hessenberg reduction inside GMRES). Acting on rows p and q it applies the LAPACK `lartg` convention

$$\begin{pmatrix} c & s \\ -\overline{s} & c \end{pmatrix} \begin{pmatrix} \mathrm{row}_p \\ \mathrm{row}_q \end{pmatrix}, \tag{1}$$

with the cosine c always real, so the rotation is unitary even for complex `T`. `FromPair` chooses (c, s) without ever squaring its inputs (`dlartg` -style scaling), so it stays exact near the overflow and underflow thresholds. `Real` is `NumericTraits<T>::Real` .

6.1 Construction

`Givens() / Givens(const T& cosine, const T& sine, Index p, Index q)`

Example

```
Givens<double> uninit;                // coefficients value-initialized
Givens<double> g(1.0, 0.0, 0, 1);     // explicit c, s and plane (p, q)
assert(g.cosine() == 1.0 && g.sine() == 0.0);
```

Description

The default constructor makes an uninitialized rotation. The four-argument constructor sets the coefficients c and s directly, together with the two plane indices p and q it acts on.

Parameters

`cosine` the cosine coefficient c
`sine` the sine coefficient s
`p` first plane index
`q` second plane index

Returns

The constructed rotation.

`static Givens FromPair(const T& a, const T& b, Index p, Index q)`

Example

```
G g = G::FromPair(3.0, 4.0, 0, 1);
V x{3, 4};
g.apply(x);
assert(close(std::abs(x(0)), 5.0) && close(x(1), 0.0)); // (a,b) -> (r,0)
assert(close(x(0), g.radius()));            // lands on the radius
assert(close(g.cosine()*g.cosine() + g.sine()*g.sine(), 1.0)); // c^2+|s|^2=1
// dlartg-style scaling: neither a^2 nor b^2 is ever materialized
G big = G::FromPair(1e200, 1e200, 0, 1);
assert(std::isfinite(big.radius()));
```

Description

Builds the rotation that maps the pair (a, b) to $(r, 0)$, i.e. the rotation that zeros the second component. The coefficients are formed with a scale factor so that neither a^2 nor b^2 is materialized, keeping the result exact near the overflow and underflow thresholds. Two edge cases: $b = 0$ gives a (possibly phase-carrying) identity, and $a = 0$ gives a pure swap.

Parameters

- **`a`** first component to rotate
- **`b`** second component (the one zeroed)
- **`p`** first plane index
- **`q`** second plane index

Returns

The rotation mapping (a, b) to $(r, 0)$; the landing radius r is available via `radius()`.

static Givens Identity(Index p, Index q)

Example

```
G g = G::Identity(0, 1);
assert(g.cosine() == 1.0 && g.sine() == 0.0);
V x{3, 4};
g.apply(x);
assert(x(0) == 3 && x(1) == 4); // no-op on the (p, q) plane
```

Description

Builds the identity rotation ($c = 1$, $s = 0$): a no-op on the (p, q) plane.

Parameters

`p`	first plane index
`q`	second plane index

Returns

The identity rotation on the (p, q) plane.

6.2 Accessors

const T& cosine() / sine() / Index firstIndex() / secondIndex() / T radius()

Example

```
G g = G::FromPair(3.0, 4.0, 0, 1);
assert(g.firstIndex() == 0 && g.secondIndex() == 1);
assert(close(std::abs(g.radius()), 5.0)); // what (a, b) rotated onto
```

Description

The rotation's stored data: the coefficients c (`cosine`) and s (`sine`); the two plane indices p (`firstIndex`) and q (`secondIndex`); and the radius r (`radius`) — the value (a, b) lands on, set by `FromPair` (zero otherwise).

Returns

A reference to c / s, the index p / q, or the radius r (by value).

6.3 Transpose & inverse

`Givens transposed()  /  Givens inverse()`

Example

```
G g = G::FromPair(3.0, 4.0, 0, 1);
V x{3, 4};
g.apply(x);
g.inverse().apply(x);                 // undo: restores the input
assert(close(x(0), 3.0) && close(x(1), 4.0));
M gd = g.toMatrix(2), td = g.transposed().toMatrix(2);
assert((gd * td).isApprox(M::Identity(2), 1e-12));
```

Description

Both return the rotation that reverses this one. Because the rotation is unitary, its `inverse` is the conjugate transpose G^H (c unchanged, $s \to -s$), while `transposed` is G^T (c unchanged, $s \to -\overline{s}$). The two coincide for real `T` and differ for complex `T`.

Returns

The transposed / inverse rotation.

6.4 Application

`void applyLeft(MatrixView<T> target) / void applyRight(MatrixView<T> target)`

Example

```
G g = G::FromPair(1.0, 1.0, 0, 2);
M a{{1, 2}, {100, 200}, {3, 4}};
g.applyLeft(a.view());
assert(a(1, 0) == 100 && a(1, 1) == 200); // row 1 (off-plane) untouched
assert(a.isApprox(g.toMatrix(3) * before, 1e-12)); // matches the dense form
// applyRight rotates the two indexed columns instead
g2.applyRight(m.view());
```

Description

Apply the rotation in place. `applyLeft` rotates rows p and q (target $\leftarrow G \cdot$ target); `applyRight` rotates columns p and q (target $\leftarrow$ target $\cdot\, G$). Each is $O(\text{cols})$ / $O(\text{rows})$: only the two indexed rows/columns are touched.

Parameters

`target` matrix view overwritten with the rotated rows / columns

Returns

Nothing (in place).

`void apply(Vector<T>& x)`

Example

```
G g = G::FromPair(3.0, 4.0, 0, 1);
V x{3, 4};
g.apply(x);
assert(close(std::abs(x(0)), 5.0) && close(x(1), 0.0));
```

Description

Applies the rotation to a vector in place, rotating entries p and q; every other entry is untouched.

Parameters

`x` vector overwritten with the rotated entries

Returns

Nothing (in place).

`Matrix<T> toMatrix(Index dimension)`

Example

```
G g = G::FromPair(3.0, -4.0, 0, 1);
M d = g.toMatrix(2);
assert(d.isOrthogonal(1e-12));       // rotation is orthogonal/unitary
assert(close(d.determinant(), 1.0)); // proper rotation
```

Description

Materializes the dense rotation: the 2×2 block embedded in an identity of the given size, with every entry off the (p, q) plane left as the identity. Intended mainly for tests and cross-checks.

Parameters

`dimension` size of the surrounding identity matrix

Returns

The dense rotation matrix.

6.5 GivensSequence

`GivensSequence<T>` is an ordered product of rotations — the accumulated sweep of a Jacobi eigenvalue pass or a bidiagonal chase. `applyLeft` applies the rotations in append order (target $\leftarrow G_{k-1} \cdots G_1 G_0 \cdot$ target); `applyLeftReversed` uses the opposite order (the inverse sequence pairs `reversed` with `transposed`).

`GivensSequence()` / `void append(const Givens<T>&)` / `clear()` / `Index count()`

Example

```
GivensSequence<double> seq;
assert(seq.count() == 0);
seq.append(G::FromPair(1.0, 2.0, 0, 1));
seq.append(G::FromPair(3.0, 1.0, 1, 2));
assert(seq.count() == 2);
seq.clear();
assert(seq.count() == 0);
```

Description

Construct an empty sequence, then grow it: `append` adds a rotation at the end, `clear` removes all rotations, `count` reports how many it holds.

Parameters

`rotation` the rotation to add (`append`)

Returns

Nothing / the rotation count (`count`).

`const Givens<T>& operator[](Index k)`

Example

```
seq.append(g0); seq.append(g1);
assert(seq[0].firstIndex() == 0 && seq[1].secondIndex() == 2);
```

Description

Read-only access to a rotation by position in the sequence.

Parameters

`k` position in the sequence (0-based)

Returns

Reference to rotation k.

`void applyLeft / applyRight / applyLeftReversed(MatrixView<T> target)`

Example

```
// applyLeft is append order: target <- G1 * (G0 * target)
M expected = g1.toMatrix(3) * (g0.toMatrix(3) * a);
M inPlace = a;
seq.applyLeft(inPlace.view());
assert(inPlace.isApprox(expected, 1e-12));
// applyLeftReversed applies them the other way round
seq.applyLeftReversed(other.view());
```

Description

Apply the whole sequence in place. `applyLeft` composes the rotations on the left in append order; `applyRight` composes them on the right in append order; `applyLeftReversed` composes on the left in reverse append order (used to apply the inverse sweep together with `transposed`).

Parameters

`target` matrix view overwritten with the result

Returns

Nothing (in place).

`GivensSequence reversed()  /  Matrix<T> toMatrix(Index dimension)`

Example

```
// toMatrix is the accumulated product; reversed() flips the order
assert(seq.toMatrix(3).isApprox(g1.toMatrix(3) * g0.toMatrix(3), 1e-12));
assert(seq.reversed().toMatrix(3).isApprox(
    g0.toMatrix(3) * g1.toMatrix(3), 1e-12));
```

Description

`reversed` returns the sequence in reverse order; `toMatrix` materializes the accumulated rotation as a dense matrix of the given size.

Parameters

`dimension` size of the identity the rotations act within (`toMatrix`)

Returns

The reversed sequence / the dense product matrix.